

源码可追溯技术文档

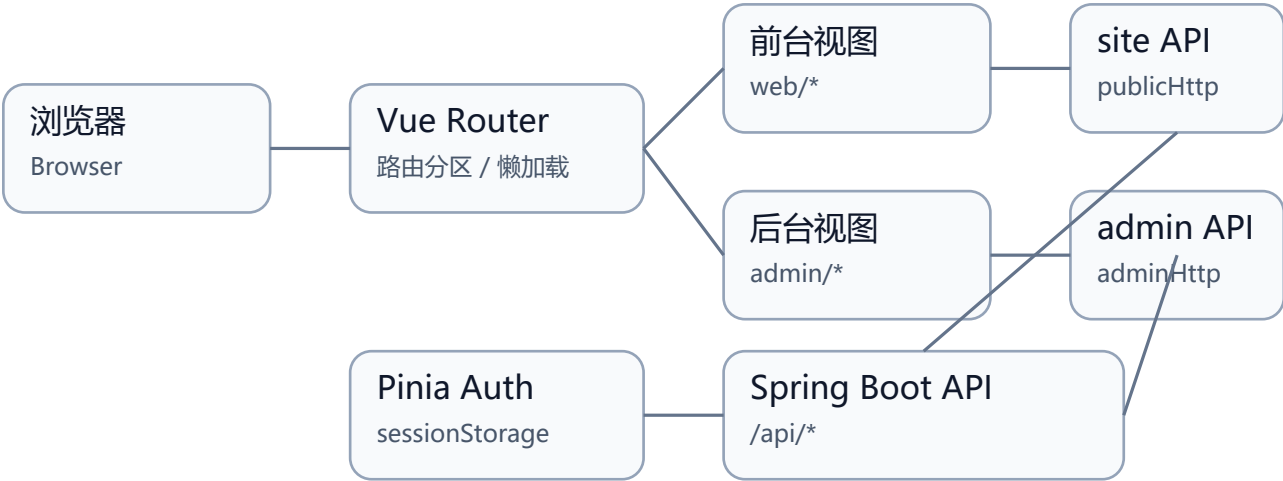
前端实现详解

来源文件: frontend.md

生成方式: Python + ReportLab (当前环境下的最小可控方案)

特点: 可点击目录、源码锚点清晰、图表内嵌、代码块可读

架构总览图



图示说明: 浏览器经由 Vue Router 分流到前台与后台视图; 后台链路额外依赖 Pinia 登录态与 adminHttp。

目录

- 目录
- 1. 项目整体架构
 - 1.1 架构总览
 - 1.2 技术选型
 - 1.3 数据流向
- 2. 模块划分与职责说明
 - 2.1 顶层模块
 - 2.2 前后台分区方式
 - 2.3 布局职责
- 3. 关键依赖与版本清单
- 4. 功能逐点拆解
 - 4.1 应用启动与基础装配
 - 4.2 前后台路由分区与懒加载
 - 4.3 后台登录态与路由守卫
 - 4.4 Axios 拦截器与统一错误处理
 - 4.5 首页聚合展示
 - 4.6 Hero 配置解析
 - 4.7 足迹地图渲染
 - 4.8 WebLayout 主题与导航可见性控制
 - 4.9 地图 SDK 延迟加载与地理数据缓存
 - 4.10 仪表盘图表与异步组件拆分
 - 4.11 后台导航与管理页组织
- 5. 数据流与状态管理
 - 5.1 组件生命周期
 - 5.2 全局状态管理
 - 5.1 组件生命周期
 - 5.2 全局状态管理
 - 5.3 路由守卫
 - 5.4 缓存策略
- 6. 安全与性能
 - 6.1 性能
 - 6.2 安全
 - 6.3 SSR/CSR 选型
- 7. 测试与质量保障
 - 7.1 测试覆盖现状
 - 7.2 自动化流水线
 - 7.3 代码规范

- 8. 部署与运维
 - 8.1 构建
 - 8.2 Nginx 托管与 API 反向代理
 - 8.3 环境变量
- 9. 现状结论与补充建议
 - 9.1 已具备能力
 - 9.2 当前未发现或未实现能力
 - 9.3 读者复现建议

目录

- [1. 项目整体架构](#1-项目整体架构)
- [2. 模块划分与职责说明](#2-模块划分与职责说明)
- [3. 关键依赖与版本清单](#3-关键依赖与版本清单)
- [4. 功能逐点拆解](#4-功能逐点拆解)
- [5. 数据流与状态管理](#5-数据流与状态管理)
- [6. 安全与性能](#6-安全与性能)
- [7. 测试与质量保障](#7-测试与质量保障)
- [8. 部署与运维](#8-部署与运维)
- [9. 现状结论与补充建议](#9-现状结论与补充建议)

1. 项目整体架构

1.1 架构总览

前端位于 frontend/，是一个基于 Vue 3 + Vite 的单页应用（SPA），前台展示站点与后台管理端共用同一套运行时，通过路由前缀与布局组件进行分区。应用入口在 frontend/src/main.ts:1-18，负责装配 Pinia、Vue Router 与 Element Plus。

核心架构图见 ../assets/frontend-architecture.mmd。

1.2 技术选型

- 框架：Vue 3 (frontend/package.json:17)
- 构建工具：Vite 6 (frontend/package.json:30)
- 路由：Vue Router 4 (frontend/package.json:20)
- 状态管理：Pinia 3 (frontend/package.json:15)
- UI 组件：Element Plus (frontend/package.json:14)
- 样式体系：Tailwind CSS + SCSS (frontend/package.json:28, frontend/package.json:27)
- 图表：ECharts + vue-echarts (frontend/package.json:13, frontend/package.json:19)
- 地图：高德地图 SDK + china-map-data (frontend/src/components/web/HomeFootprintMap.vue:126-127)

1.3 数据流向

典型数据流为：

View -> API -> Axios -> /api -> Spring Boot -> -> View

对于后台接口，请求会额外经过登录态仓库，自动注入 Bearer Token，见 frontend/src/api/http.ts:53-59。

2. 模块划分与职责说明

2.1 顶层模块

1. src/views/web/：前台公开页面，例如首页、博客、关于页、留言页、动态页。

- 1. src/views/admin/: 后台管理页面，例如仪表盘、媒体管理、文章管理、系统配置。
- 1. src/layouts/: 前后台布局骨架。
- 1. src/router/: 路由定义与守卫。
- 1. src/stores/: 全局状态，目前主要用于后台认证状态。
- 1. src/api/: 按公共站点与后台管理拆分 API 调用。
- 1. src/utils/: 地图、日期、媒体等工具函数。
- 1. src/styles/: 全局样式与设计 token。

2.2 前后台分区方式

路由定义位于 frontend/src/router/index.ts:9-99:

- / 下挂载 WebLayout，承载公开站点页面。
- /dyx-manager 下挂载 AdminLayout，承载后台管理页面。
- /dyx-manager/login 作为独立登录页。

这种设计的直接收益是：

- 部署上只需一套静态资源。
- 共享公共能力（HTTP、样式、构建配置）。
- 后台管理模块仍可用路由元信息进行隔离。

2.3 布局职责

- frontend/src/layouts/WebLayout.vue：承载前台导航、主题与公开内容容器。
- frontend/src/layouts/AdminLayout.vue:1-235：提供后台导航分组、头部退出登录按钮、移动端抽屉导航。

AdminLayout.vue:136-177 把后台功能组织为“概览 / 首页管理 / 内容管理 / 个人资料 / 系统”五组，使导航与后端模块划分基本一致。

3. 关键依赖与版本清单

依赖	版本	作用
vue	^3.5.13	前端核心框架
vite	^6.3.5	构建与开发服务器
vue-router	^4.5.1	路由系统
pinia	^3.0.2	登录态全局状态
element-plus	^2.10.7	后台与表单组件
echarts	^6.0.0	仪表盘图表展示
vue-echarts	^8.0.1	Vue 图表封装
tailwindcss	^3.4.17	原子化样式
sass	^1.98.0	SCSS 预处理
axios	^1.9.0	HTTP 客户端
china-map-data	^1.0.4	足迹地图边界数据

来源: frontend/package.json:1-33

4. 功能逐点拆解

4.1 应用启动与基础装配

场景

应用启动时需要一次性挂载状态管理、路由系统与 UI 组件库。

需求

保证所有页面共享同一套路由、全局样式和后台登录态。

实现方案

在 frontend/src/main.ts:12-18 中创建 Vue 应用实例，并依次注册 Pinia、Router、Element Plus。

选型理由

该实现足够直接，没有引入额外框架包装层，便于维护单仓库博客场景。

核心源码定位

- frontend/src/main.ts:12-18

ts

```
const dyxApp = createApp(App);
const dyxPinia = createPinia();

dyxApp.use(dyxPinia);
dyxApp.use(router);
dyxApp.use(ElementPlus);
dyxApp.mount('#app');
```

替代方案对比

- 替代方案：在 App.vue 内部再二次注入插件。
- 弃用原因：插件装配属于应用入口职责，放在 main.ts 更清晰，也更符合 Vue 社区习惯。

4.2 前后台路由分区与懒加载

场景

站点同时存在公开页面与后台管理页面，需要逻辑隔离但共用同一应用。

需求

通过路由前缀区分权限域，并尽量减少首屏包体积。

实现方案

frontend/src/router/index.ts:9-77 定义 webRoutes 与 adminRoutes 两组路由；各页面组件通过 () => import(...) 动态导入，实现路由级懒加载。

选型理由

- 分区后结构直观。
- 动态导入可以降低初始包体积，后台页面不会阻塞前台首屏。

核心源码定位

- frontend/src/router/index.ts:9-27
- frontend/src/router/index.ts:35-68
- frontend/src/router/index.ts:74-77

ts

```
const router = createRouter({
  history: createWebHistory(),
  routes: [...webRoutes, ...adminRoutes]
});
```

替代方案对比

- 替代方案：全部页面静态导入。
- 弃用原因：后台页面较多，静态导入会拉高首屏包体积；当前博客站点更适合按路由切包。

4.3 后台登录态与路由守卫

场景

后台页面必须在登录后访问，未登录时需要跳转到登录页，并保留回跳地址。

需求

- 登录成功后保存 token 和用户信息。
- 未登录访问受保护页面时自动跳转。
- 已登录再次访问登录页时直接进入仪表盘。

实现方案

1. frontend/src/stores/auth.ts:26-61 使用 Pinia 保存 token 与用户信息。
1. 状态持久化到 sessionStorage，键名为 dyx-admin-token 与 dyx-admin-user。
1. frontend/src/router/index.ts:83-97 使用 beforeEach 守卫拦截带 requiresAuth 的后台路由。

选型理由

- Pinia 足够轻量，适合当前只管理认证状态的场景。
- 使用 sessionStorage 而非 localStorage，更偏向“浏览器会话级后台登录”，降低长期残留风险。

核心源码定位

- frontend/src/stores/auth.ts:14-59
- frontend/src/router/index.ts:83-97

ts

```
if (to.meta.requiresAuth && !authStore.isAuthenticated) {
  return {
    path: '/dyx-manager/login',
    query: { redirect: to.fullPath }
  };
}
```

替代方案对比

- 替代方案：纯内存态，不持久化。
- 弃用原因：页面刷新后会立刻丢失登录态，后台体验较差。
- 替代方案：持久化到 localStorage。
- 弃用原因：登录有效期更长，但后台管理场景下安全边界更松；当前代码显式清理 localStorage，说明设计偏向会话级存储（frontend/src/stores/auth.ts:45-58）。

4.4 Axios 拦截器与统一错误处理

场景

需要统一处理 API 前缀、后台 Token 注入，以及 401/403 错误反馈。

需求

- 前后台 API 共用 /api 前缀。
- 后台接口自动附带 Bearer Token。
- 登录过期时清空状态并跳转登录页。

实现方案

frontend/src/api/http.ts:8-79 创建 publicHttp 与 adminHttp 两个 Axios 实例：

- 两者共享 baseUrl：'/api' 与超时配置。
- adminHttp 通过请求拦截器自动注入 Authorization。
- 响应拦截器把非 code === 200 的统一返回结构转换为 Promise reject。
- 遇到 401 时通过 redirectToLogin() 清空登录态并跳转。

选型理由

把后台行为封装在独立实例中，避免公开接口被无意义地附加认证逻辑。

核心源码定位

- frontend/src/api/http.ts:8-12
- frontend/src/api/http.ts:34-47
- frontend/src/api/http.ts:53-59
- frontend/src/api/http.ts:65-77

ts

```
adminHttp.interceptors.request.use((config) => {
  const authStore = useAuthStore();
  if (authStore.token) {
    config.headers.Authorization = `Bearer ${authStore.token}`;
  }
  return config;
});
```

替代方案对比

- 替代方案：在每个 API 函数手工拼接 Authorization 请求头。
- 弃用原因：重复代码多，且容易漏掉新增接口。

4.5 首页聚合展示

场景

首页需要一次性展示 Hero 区、最新博客、最新动态、精选项目、最新荣誉与足迹地图。

需求

减少首屏接口往返次数，并保持首页数据结构稳定。

实现方案

- frontend/src/views/web/HomeView.vue:251-259 通过 `getHomeData()` 拉取聚合首页数据。
- HomeData 类型在 frontend/src/api/modules/site.ts:172-180 定义。
- 页面内部再根据 heroConfig、footprints 等字段拆解为具体渲染块。

选型理由

首页是典型聚合场景，使用单接口拉取比页面层多次拼装请求更高效，也更容易让前后端围绕同一 DTO 迭代。

核心源码定位

- frontend/src/api/modules/site.ts:169-180
- frontend/src/views/web/HomeView.vue:114-121
- frontend/src/views/web/HomeView.vue:241-260

替代方案对比

- 替代方案：博客、项目、荣誉、足迹分别独立请求。
- 弃用原因：请求数更多，首页加载链路更长，且页面层需要协调多个 loading/error 状态。

4.6 Hero 配置解析

场景

首页首屏不是固定模板，而是允许后台通过配置调整眉题、标题、副标题、标签和图片。

需求

- 配置缺失时也能安全回退。
- 需要保持页面结构可预测。

实现方案

frontend/src/api/modules/site.ts:196-255 提供：

- `createDefaultHeroConfig()`：生成默认配置。
- `parseHeroConfig()`：解析 profile 中的 JSON 配置，失败则回退默认值。

选型理由

把配置解析逻辑放在 API/types 层，可以让 View 层专注于渲染，而不是处理 JSON 容错。

核心源码定位

- frontend/src/api/modules/site.ts:196-255

ts

```
try {
  const parsed = JSON.parse(profile.heroConfig) as Partial<HeroConfigData>;
  if (!parsed || !Array.isArray(parsed.blocks)) {
    return fallback;
  }
  return {
    version: typeof parsed.version === 'number' ? parsed.version : 1,
    blocks: parsed.blocks as HeroBlock[]
  };
} catch {
  return fallback;
}
```

替代方案对比

- 替代方案：页面组件内部直接 `JSON.parse`。
- 弃用原因：多个页面或后台表单若都要解析，会造成逻辑分散；当前实现把容错收敛到了类型层。

4.7 足迹地图渲染

场景

首页第二屏需要用地图展示地理足迹，并兼顾移动端与桌面端交互提示。

需求

- 支持深浅主题。
- 允许高德地图底图与自定义边界/标签叠加。
- 在缩放级别变化时动态控制国家名、城市名、边界显示。

实现方案

frontend/src/components/web/HomeFootprintMap.vue:112-359 使用高德地图 SDK 与本地地理数据工具：

- 通过 `loadAmapSdk()` 按需加载 SDK。
- 使用 `featureToPolygonPaths`、`getMapBoundaryItems`、`getWorldCountryLabelItems` 等工具生成覆盖物。
- 根据 `zoom` 阈值动态控制国家标签、城市边界、已访问城市标签显隐。

选型理由

地图是高度定制化模块，直接基于 AMap + 本地边界数据做二次绘制，比通用图表库更容易实现足迹高亮、标签策略和视觉定制。

核心源码定位

- frontend/src/components/web/HomeFootprintMap.vue:113-127
- frontend/src/components/web/HomeFootprintMap.vue:197-205
- frontend/src/components/web/HomeFootprintMap.vue:319-331

ts

```
function ensureWorldLayer(AMap: AMapNamespace): void {
  if (!mapInstance || !AMap.DistrictLayer?.World) {
    return;
  }
  if (!worldLayer) {
    worldLayer = new AMap.DistrictLayer.World({
      zIndex: 1,
      zooms: [MIN_ZOOM, MAX_ZOOM],
    });
    worldLayer.setMap(mapInstance);
  }
  worldLayer.setStyles(getWorldLayerStyles());
}
```

替代方案对比

- 替代方案：用静态图片或纯 SVG 地图渲染。
- 弃用原因：交互缩放、拖拽、城市级标注能力较弱，难以达到当前视觉效果。

4.8 WebLayout 主题与导航可见性控制

场景

公开站点既要支持深浅主题切换，又要在首页全屏滚动场景中控制顶部导航显隐。

需求

- 支持自动主题与手动主题切换。
- 主题选择应具备持久化能力。
- 首页与普通内容页的布局行为不同。

实现方案

frontend/src/layouts/WebLayout.vue:17-145 负责：

1. 使用 provide() 向子组件暴露主题与导航可见性控制函数。
1. 通过 localStorage 读取/保存用户手动主题选择。
1. 在没有手动主题时，按 8:00~18:00 使用浅色、其余时间使用深色。
1. 根据当前路由是否为首页，切换全屏滚动布局与普通文档布局。

选型理由

把主题和顶栏显隐集中在

WebLayout，比散落在每个页面中更容易保持全站一致性，也避免多页面重复实现主题切换。

核心源码定位

- frontend/src/layouts/WebLayout.vue:17-33
- frontend/src/layouts/WebLayout.vue:35-49
- frontend/src/layouts/WebLayout.vue:64-118
- frontend/src/layouts/WebLayout.vue:121-145

ts

```
const layoutClass = computed(() =>
  isHomeRoute.value ? 'h-screen overflow-hidden' : 'min-h-screen overflow-x-hidden'
);

function toggleTheme(): void {
  const nextTheme: ThemeMode = theme.value === 'dark' ? 'light' : 'dark';
  syncDocumentTheme(nextTheme);
  window.localStorage.setItem(THEME_STORAGE_KEY, nextTheme);
  clearScheduledThemeTimer();
}
```

替代方案对比

- 替代方案：每个页面自己维护主题状态。
- 弃用原因：容易出现主题不一致和重复逻辑，尤其是首页与普通页布局差异较大。

4.9 地图 SDK 延迟加载与地理数据缓存

场景

足迹地图属于重模块，不适合阻塞首页首屏；同时地理边界数据解码成本较高，需要避免重复计算。

需求

- 高德地图脚本只在真正需要时加载。
- 地理边界数据只解码一次，多次缩放和切换时复用结果。
- 缺失 API Key 时应明确失败，而不是静默异常。

实现方案

- frontend/src/utils/amapLoader.ts:24-87 使用 Promise 缓存 SDK 加载过程，避免重复插入脚本。
- frontend/src/utils/footprintGeo.ts:166-181 对中国、省级、世界 GeoJSON 做规范化与缓存。
- boundaryCache 与 normalizedProvinceCache 用于复用已经处理过的边界数据。

选型理由

这类地图模块的性能瓶颈不在普通页面渲染，而在 SDK 下载与地理数据解码。当前做法把两者都延迟并缓存，是非常符合场景的优化。

核心源码定位

- frontend/src/utils/amapLoader.ts:24-87
- frontend/src/utils/footprintGeo.ts:140-181

ts

```
if (amapLoaderPromise) {
  return amapLoaderPromise;
}

const normalizedProvinceCache = new Map<string, GeoJsonCollection>();
const boundaryCache = new Map<string, GeoJsonFeature[]>();
```

替代方案对比

- 替代方案：应用启动时就加载地图 SDK。

- 弃用原因：会拖慢不访问首页足迹模块的用户首屏。
- 替代方案：每次进入地图都重新解析 GeoJSON。
- 弃用原因：重复 CPU 开销大，缩放和切页体验会变差。

4.10 仪表盘图表与异步组件拆分

场景

后台仪表盘需要展示摘要卡片、趋势图和热门页面排行，但图表库本身较重。

需求

- 首屏先渲染摘要信息。
- 图表模块按需加载，避免占用后台所有页面的初始包体积。
- 统计卡片需要从同一份摘要数据派生。

实现方案

frontend/src/views/admin/AdminDashboardView.vue:75-215 采用：

- defineAsyncComponent() 延迟加载 AdminDashboardCharts.vue。
- summary 作为单一数据源，再派生 trendPoints、deviceStats、topPages、summaryCards。
- onMounted() 调用 getDashboardSummary() 拉取摘要数据。

选型理由

仪表盘是后台中的“重量页面”，但图表能力又只在该页面需要。异步拆分组件比把图表逻辑混进全局入口更节省资源。

核心源码定位

- frontend/src/views/admin/AdminDashboardView.vue:76-95
- frontend/src/views/admin/AdminDashboardView.vue:97-172
- frontend/src/views/admin/AdminDashboardView.vue:201-215

ts

```
const AdminDashboardCharts = defineAsyncComponent(
  () => import('@/components/admin/AdminDashboardCharts.vue')
);

onMounted(() => {
  void loadDashboardSummary();
});
```

替代方案对比

- 替代方案：图表组件静态导入。
- 弃用原因：所有后台页面都会为仪表盘图表付出初始加载成本。

4.11 后台导航与管理页组织

场景

后台功能较多，需要稳定导航结构，让内容管理与系统管理分区清晰。

需求

支持桌面端侧栏与移动端抽屉导航，并保持路由驱动菜单高亮。

实现方案

frontend/src/layouts/AdminLayout.vue:136-182 用数组声明导航组，再交给 Element Plus 菜单组件渲染。

选型理由

数据驱动菜单比手写大量模板更易维护，新增页面时只改一处即可。

核心源码定位

- frontend/src/layouts/AdminLayout.vue:136-177
- frontend/src/layouts/AdminLayout.vue:179-182

替代方案对比

- 替代方案：在模板中逐个写死菜单项。
- 弃用原因：层级多、重复代码多，不利于维护后台导航分组。

5. 数据流与状态管理

5.1 组件生命周期

首页页面在挂载时会获取聚合数据、组装足迹视图模型，再根据滚动位置控制不同 section。HomeView.vue 还负责把足迹相关数据映射为地图可消费结构。

后台仪表盘则在 AdminDashboardView.vue:201-215 挂载时加载摘要数据，再通过多个 computed 派生卡片、趋势与排行视图。

5.2 全局状态管理

5.1 组件生命周期

首页页面在挂载时会获取聚合数据、组装足迹视图模型，再根据滚动位置控制不同 section。HomeView.vue 还负责把足迹相关数据映射为地图可消费结构。

5.2 全局状态管理

当前全局状态集中在 frontend/src/stores/auth.ts:9-61，范围较克制：

- 只保存后台认证信息。
- 业务数据（博客、动态、留言等）按页面请求，不做复杂全局缓存。

这说明前端设计偏向“页面拉取 + 局部状态”，而不是 Redux/Vuex 风格的大型全局仓库。

5.3 路由守卫

frontend/src/router/index.ts:83-97 使用前置守卫完成：

- 未登录拦截后台访问。
- 登录后拦截重复进入登录页。
- 通过 query 保存重定向目标。

5.4 缓存策略

前端未发现复杂客户端缓存层：

- 登录态缓存：sessionStorage
- 业务数据缓存：主要依赖后端 @Cacheable，前端未见 SW/IndexedDB/Query 缓存框架。

6. 安全与性能

6.1 性能

资源拆包

frontend/vite.config.ts:4-33 通过 manualChunks 手工分包：

- vendor-core: Vue / Router / Pinia
- vendor-ui: Element Plus
- vendor-charts: ECharts
- vendor-map: 地图数据与足迹工具

这是一个非常明确的性能优化信号，说明项目已针对地图、图表等重依赖做包体切分。

路由懒加载

路由组件全部采用动态导入，见 frontend/src/router/index.ts:15-26 与 frontend/src/router/index.ts:37-66。

6.2 安全

Token 注入

后台接口统一从 Pinia 读取 token 并注入 Authorization，见 frontend/src/api/http.ts:53-59。

登录失效处理

401 时会清理认证状态并强制跳转登录页，避免前端继续带失效 token 工作，见 frontend/src/api/http.ts:65-77。

Token 刷新机制

当前未发现 Refresh Token 或无感刷新实现，文档中应明确说明“未实现”。

6.3 SSR/CSR 选型

当前为典型 CSR 架构：

- 入口为 index.html
- 使用 createWebHistory()
- 由 Nginx try_files 支持前端路由刷新

未发现 SSR/Nuxt 相关配置，因此该项目明确是 CSR。

7. 测试与质量保障

7.1 测试覆盖现状

当前仓库未发现以下内容：

- 前端单元测试文件
- E2E 测试目录
- Vitest / Jest / Cypress / Playwright 配置

因此该部分在最终交付中应写为：****当前仓库未纳入前端自动化测试基建****。

7.2 自动化流水线

当前仓库未发现：

- .github/workflows/
- GitLab CI 配置
- Jenkinsfile

因此不能将前端 CI/CD 写成已具备能力。

7.3 代码规范

当前仓库未发现 ESLint、Prettier 配置文件；可观察到代码整体保持 TypeScript + Vue SFC 结构化写法，但缺少可执行的静态检查配置。

8. 部署与运维

8.1 构建

前端构建命令来自 frontend/package.json:5-8:

```
json
    "build": "vue-tsc --noEmit && vite build"
```

8.2 Nginx 托管与 API 反向代理

docs/nginx.conf:4-42 说明了生产部署方式：

- / 与 /dyx-manager/ 指向前端静态资源目录。
- /api/dyx-manager/ 与 /api/ 反向代理到 localhost:8080。
- try_files \$uri \$uri/ /index.html; 负责支持 Vue Router 刷新。

8.3 环境变量

前端存在：

- frontend/.env.local
- frontend/.env.production

但本次文档编写基于源码与公开配置结构，不对未读取内容做臆测说明。

9. 现状结论与补充建议

9.1 已具备能力

- 前后台一体化 SPA 架构

- 路由分区与后台鉴权守卫
- Pinia 登录态管理
- Axios 拦截器与统一失败跳转
- Vite 手工分包
- 首页聚合接口消费
- 高定制足迹地图渲染
- Nginx 反向代理部署方案

9.2 当前未发现或未实现能力

- SSR
- Token 自动刷新
- 前端单测 / E2E
- ESLint / Prettier
- CI/CD 流水线
- 文档 lint 配置

9.3 读者复现建议

若读者要在 10 分钟内定位源码，建议优先按以下顺序阅读：

1. frontend/src/main.ts
1. frontend/src/router/index.ts
1. frontend/src/stores/auth.ts
1. frontend/src/api/http.ts
1. frontend/src/api/modules/site.ts
1. frontend/src/views/web/HomeView.vue
1. frontend/src/components/web/HomeFootprintMap.vue
1. frontend/src/layouts/AdminLayout.vue